High Availability Web Infrastructure & Scalable App Services

1. Project Overview

This project focused on building a resilient, scalable web architecture in Microsoft Azure. It involved two primary architectural patterns:

1. **IaaS High Availability:** Utilized an Azure Load Balancer to distribute traffic across a backend pool of virtual machines with health monitoring.
2. **PaaS Modernization:** Deployed a Web App (App Service) with advanced features like Auto-scaling, Deployment Slots, and Identity-based Security.

DNS Management: Public and Private DNS namespaces were configured and managed to ensure seamless resource resolution.

2. Tools & Technologies Used

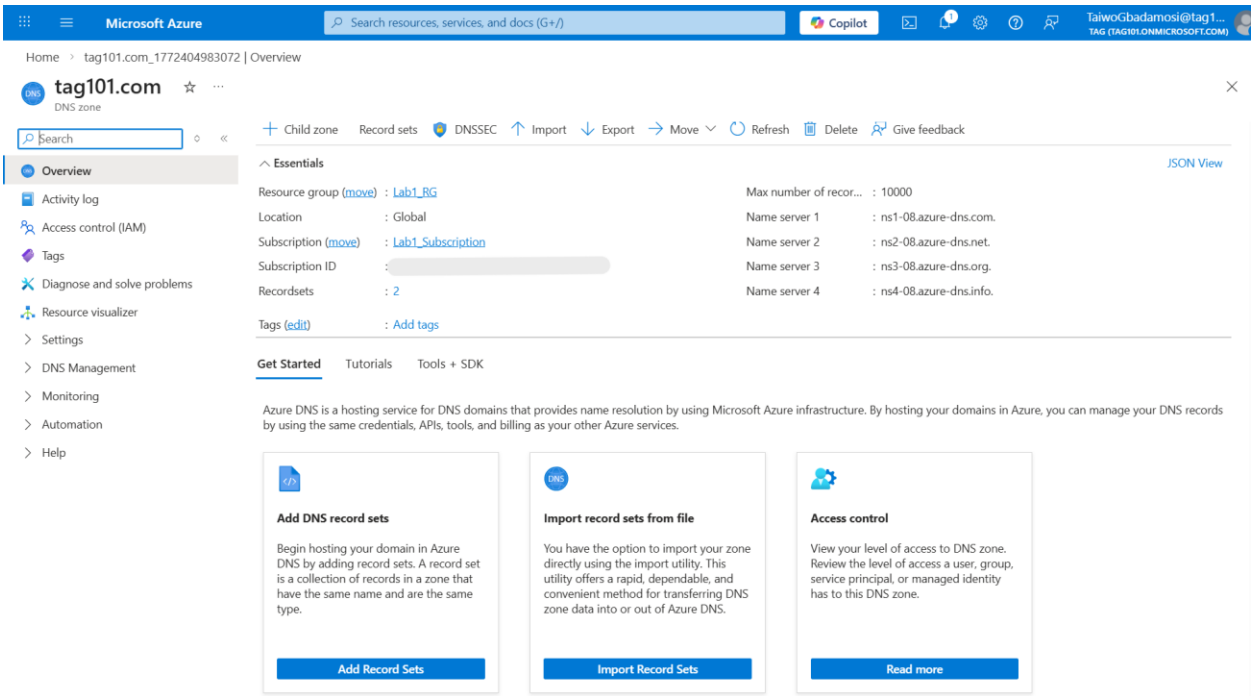
- **Networking:** Azure Load Balancer (Standard SKU), Public/Private DNS Zones, A Records, Health Probes.
- **Compute:** Azure Virtual Machines (IIS Web Servers), App Service Plans.
- **DevOps/Deployment:** Deployment Slots (Staging/Production), Swap Operations.
- **Security:** App Service Authentication.
- **Scalability:** Azure Monitor Autoscale (CPU-based rules).

3. Detailed Technical Implementation

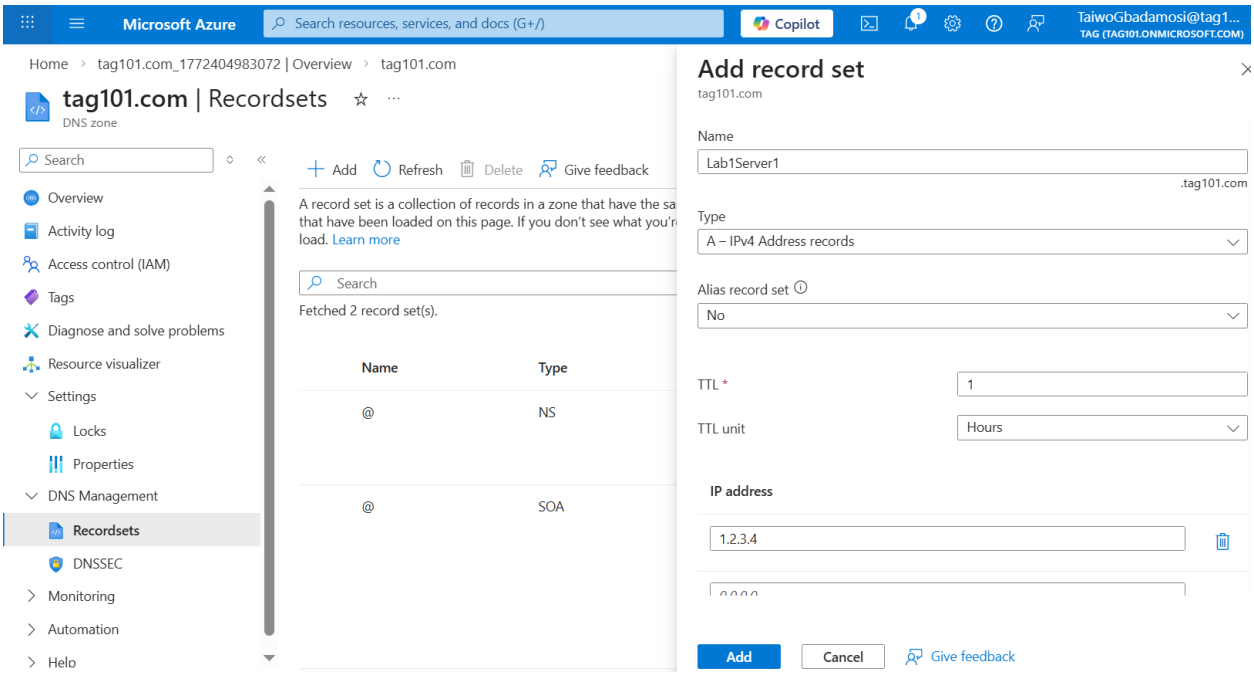
Section I: DNS Hierarchy & Resolution

I established the naming foundation by configuring both Public and Private DNS zones. This included creating "A Records" to map human-readable domain names to static IP addresses.

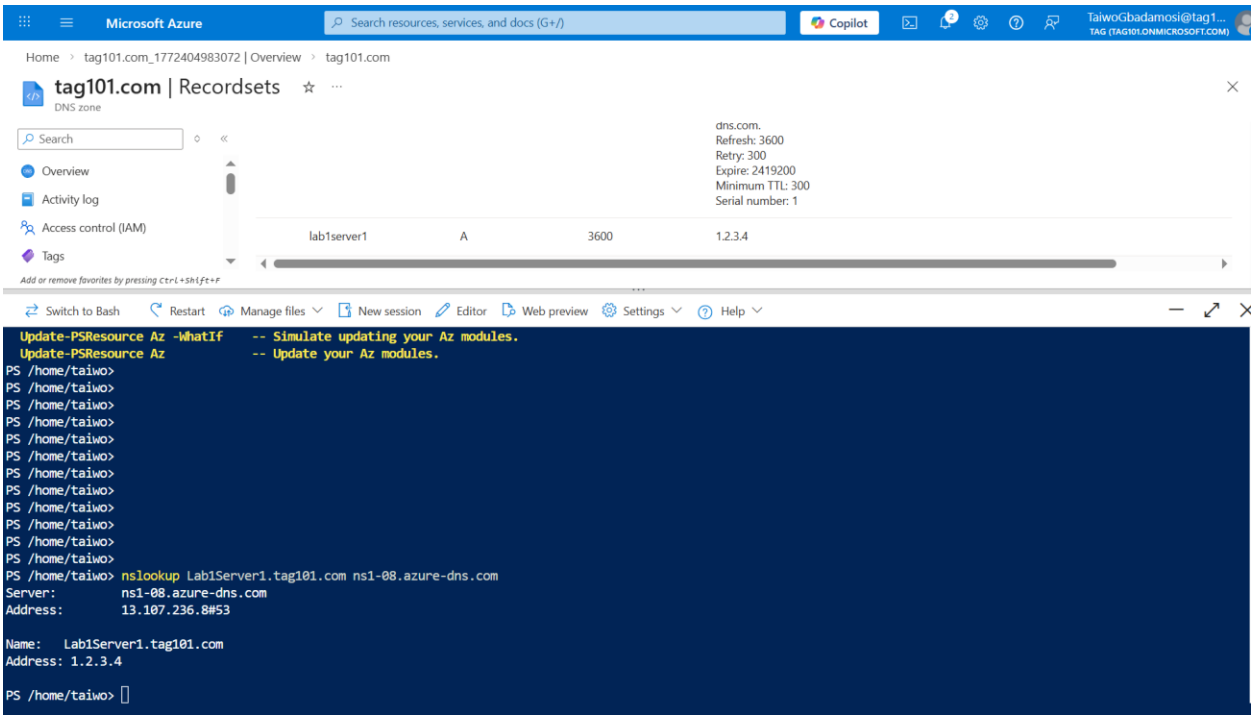
- **Verification:** Used PowerShell/Command Prompt to perform nslookup tests, confirming that the DNS zones correctly resolved to the intended backend IPs even before global registrar delegation.



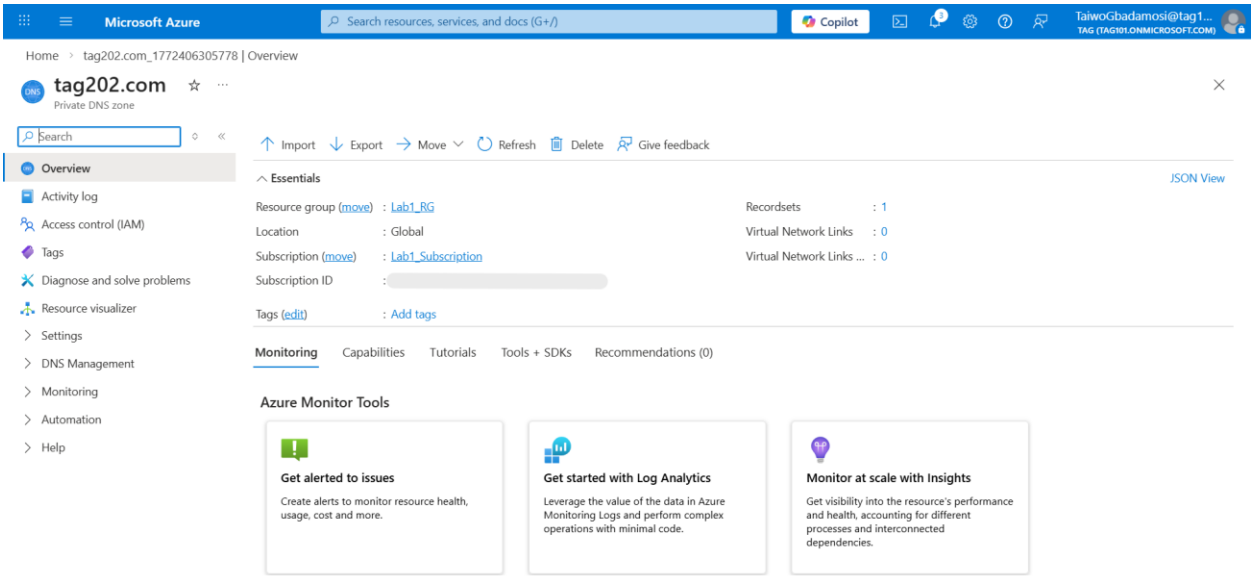
creating a public DNS zone (unregistered)



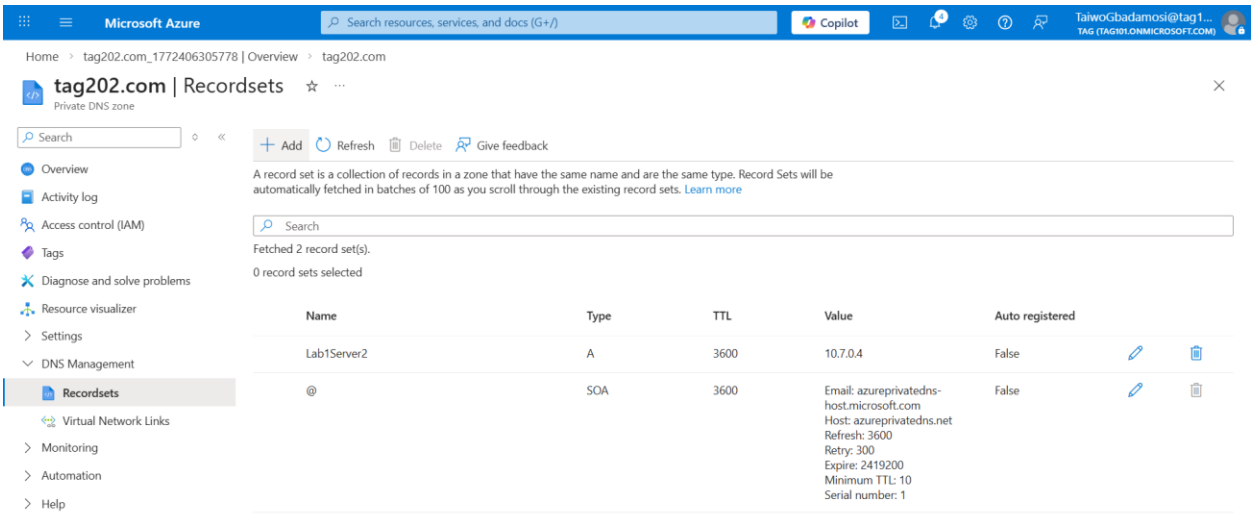
creating an A record to resolve DNS to a sample ip address (1.2.3.4)



nslookup validating DNS to ip address (Azure ns)



creating a private DNS zone

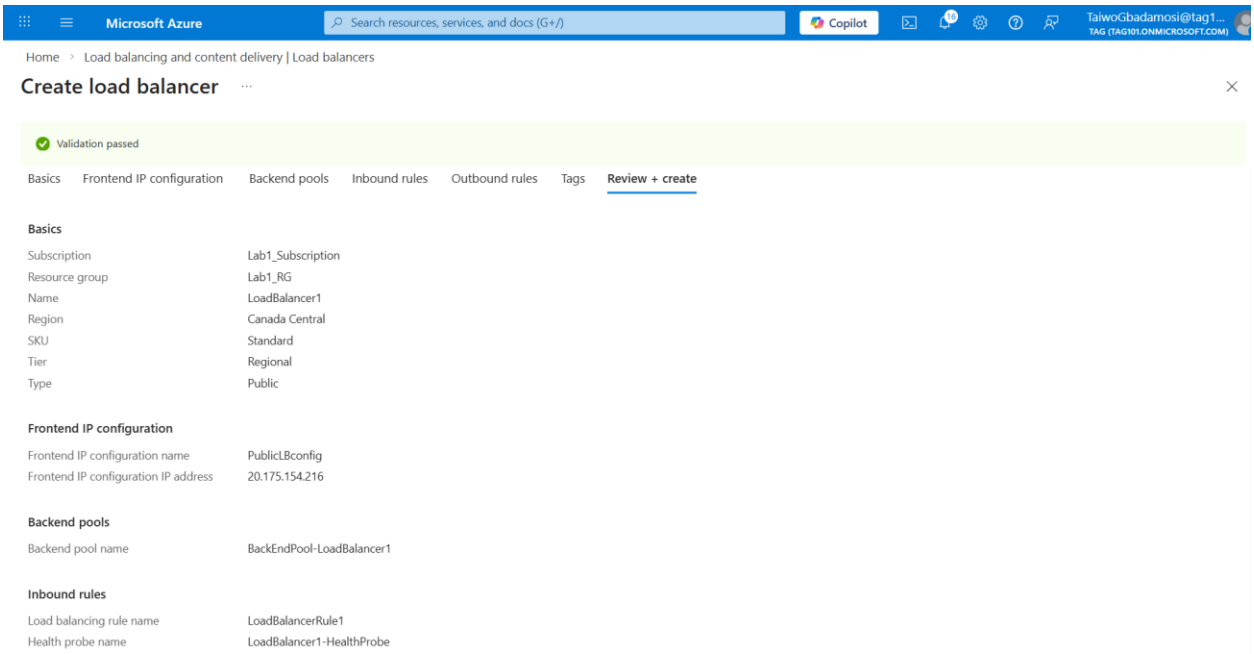


configuring private DNS to resolve to ip address (10.7.0.4)

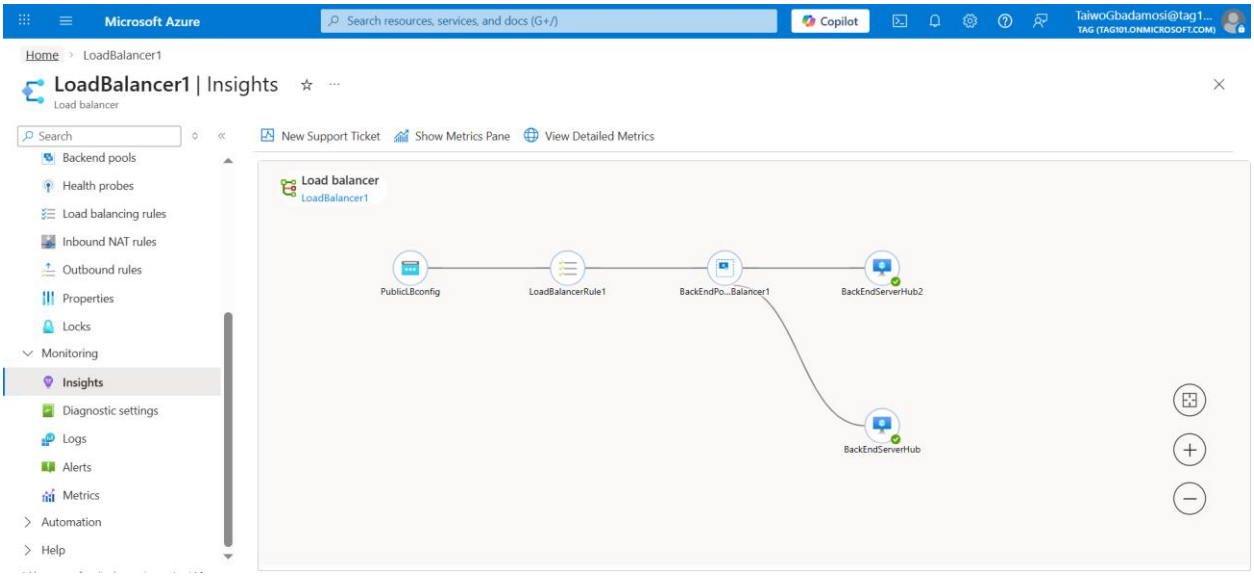
Section II: Load Balancer & High Availability

To ensure zero downtime, I implemented a Public Load Balancer to act as a single entry point for web traffic. Session Persistence was enabled which effectively keeps a user on the same server during their session.

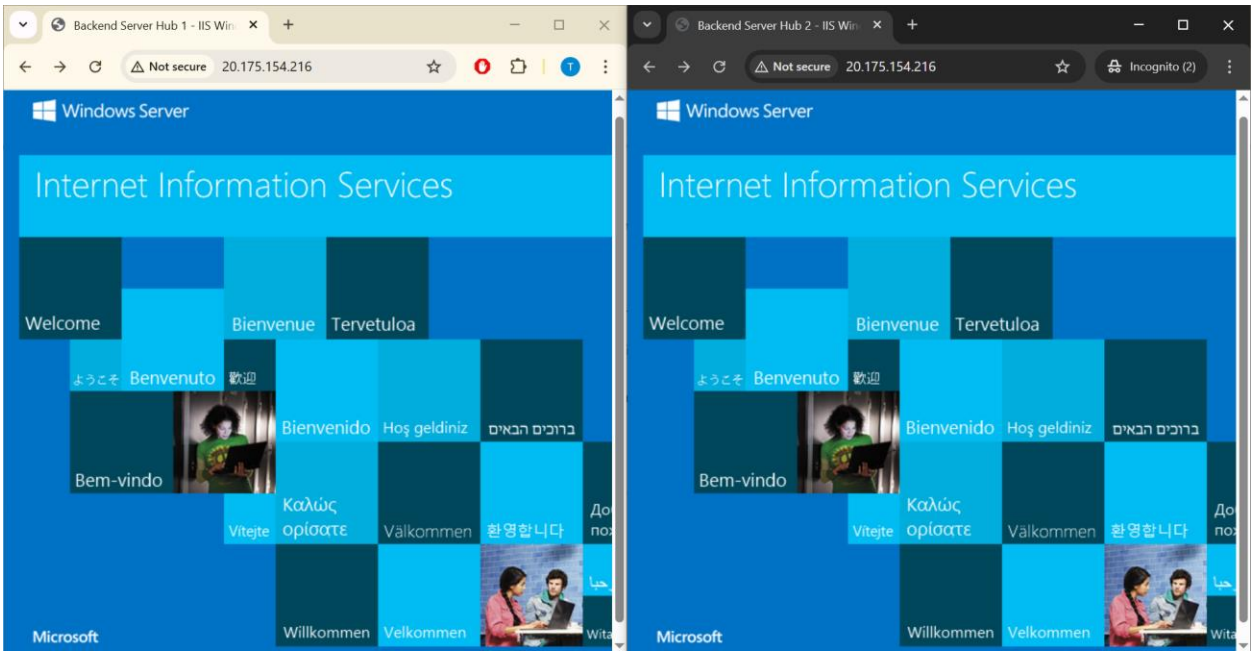
- **Backend Pools:** Grouped two Virtual Machines to handle incoming requests.
- **Health Probes:** Configured a TCP/HTTP probe to monitor the heartbeat of the servers.
- **Load Balancing Rules:** Set up rules to map frontend port 80 to the backend pool.
- **Validation:** Demonstrated that the Load Balancer correctly identified both servers as Healthy and successfully distributed traffic between them when accessing the Load Balancer's Public IP.



creating and configuring load balancer



load balancer insights showing connection between load balancer, rule, backend pools and healthy status backend pool instances

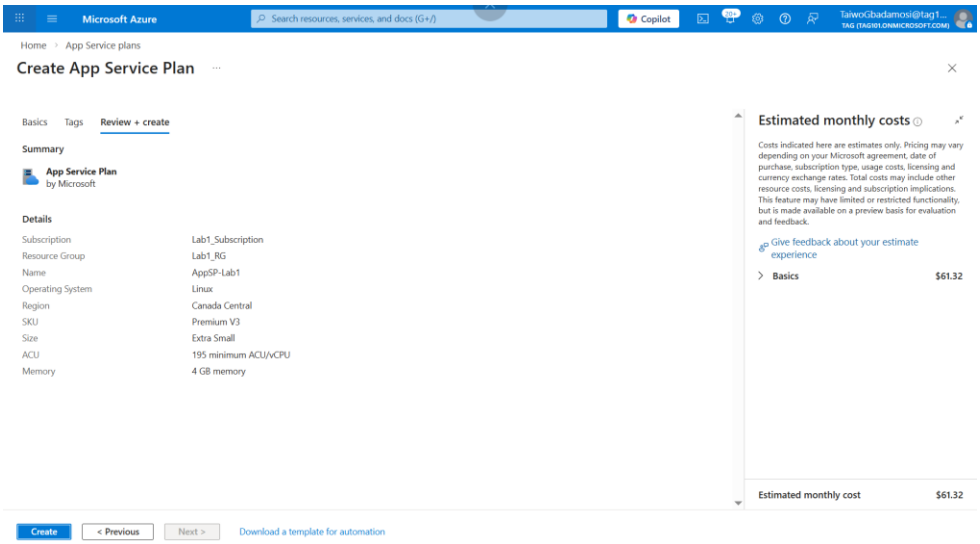


load balancer successfully resolving to both backend pool web server instances

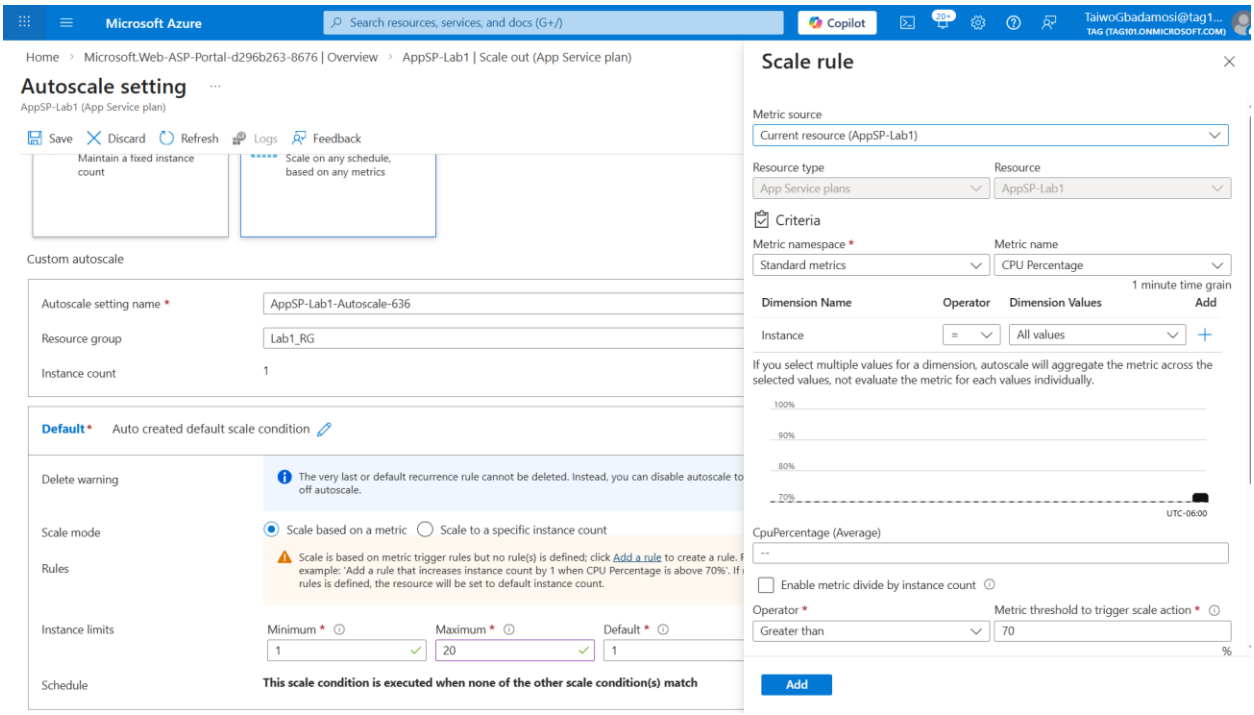
Section III: App Service & Scalable Web Apps

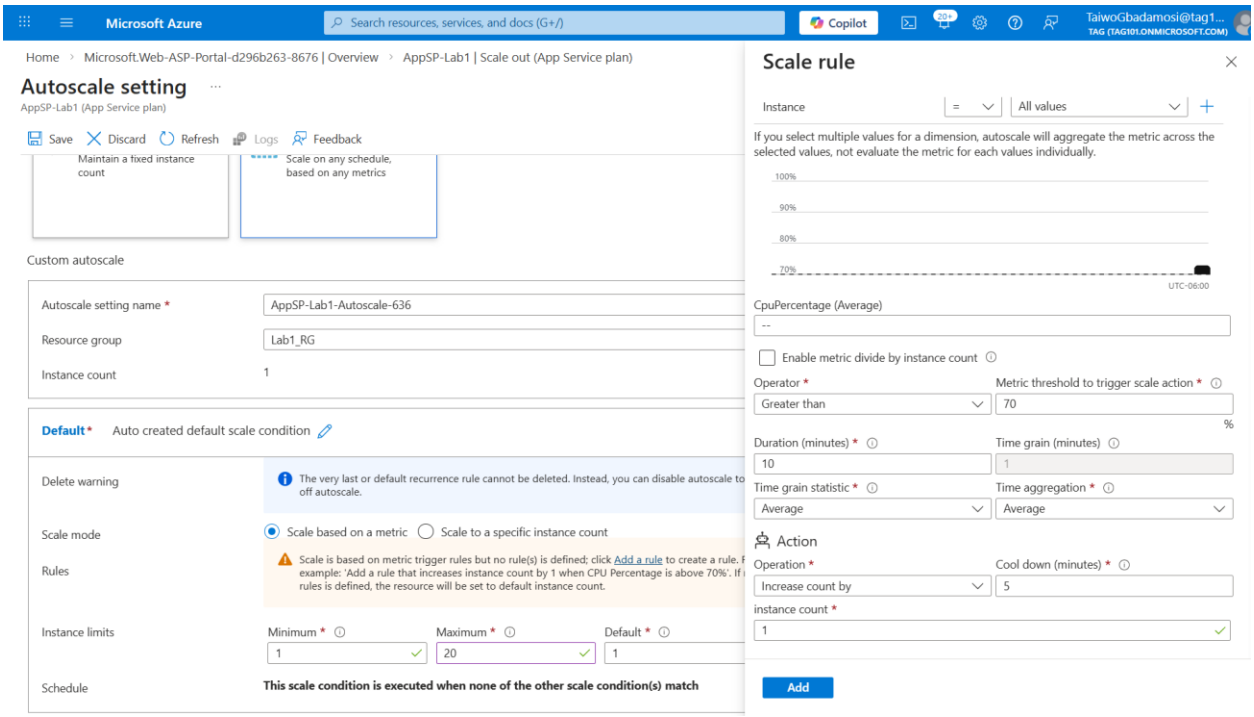
I transitioned to a Platform-as-a-Service (PaaS) model by deploying an Azure Web App.

- **App Service Plan:** Configured the underlying compute power and enabled Autoscale Rules (scale out by 1 instance if CPU > 70% for 10 minutes with a cool down period of 5 minutes).
- **Global Access:** Verified the web app was live on the internet via its default .azurewebsites.net https URL.

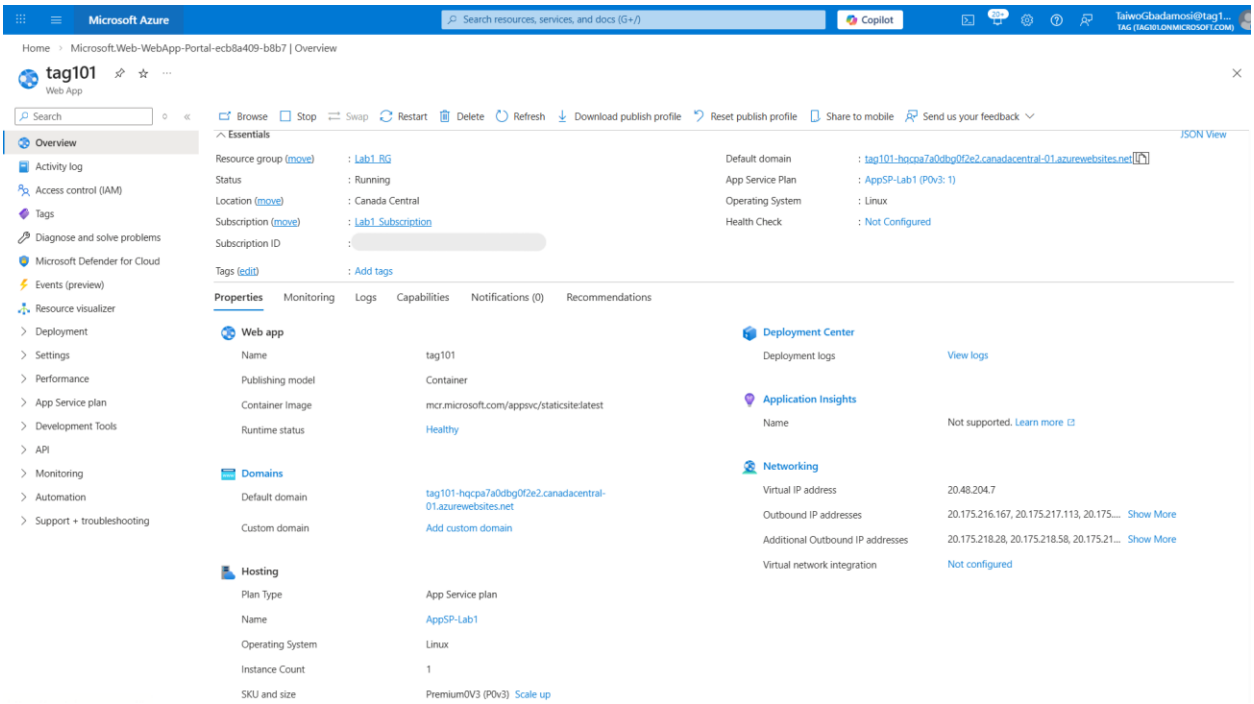


creating an app service plan

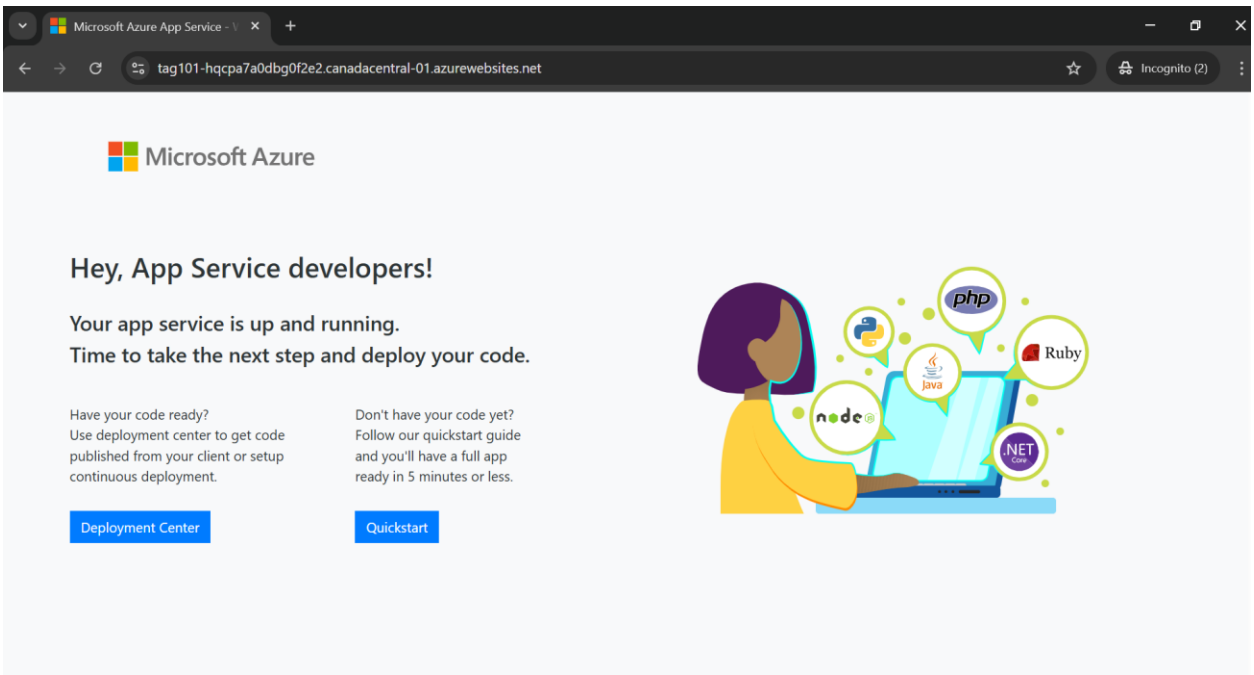




configuring app service plan autoscale function for scaling out via cpu performance percentage



creating a new app service web app with assigned default domain (azurewebsites.net)

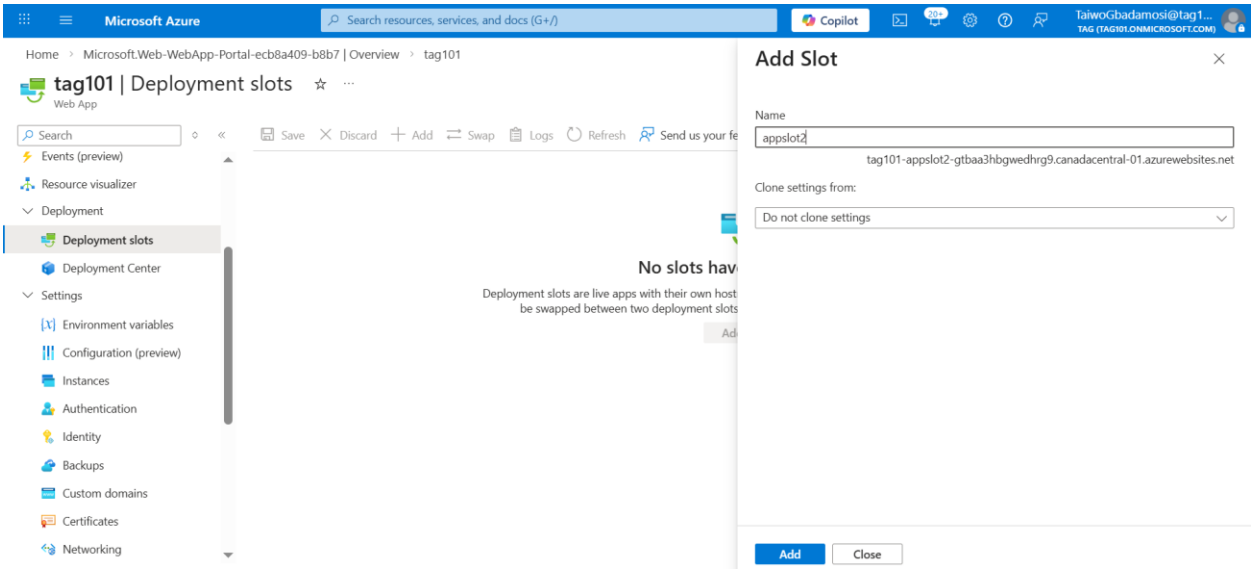


app service web app resolvable from the internet

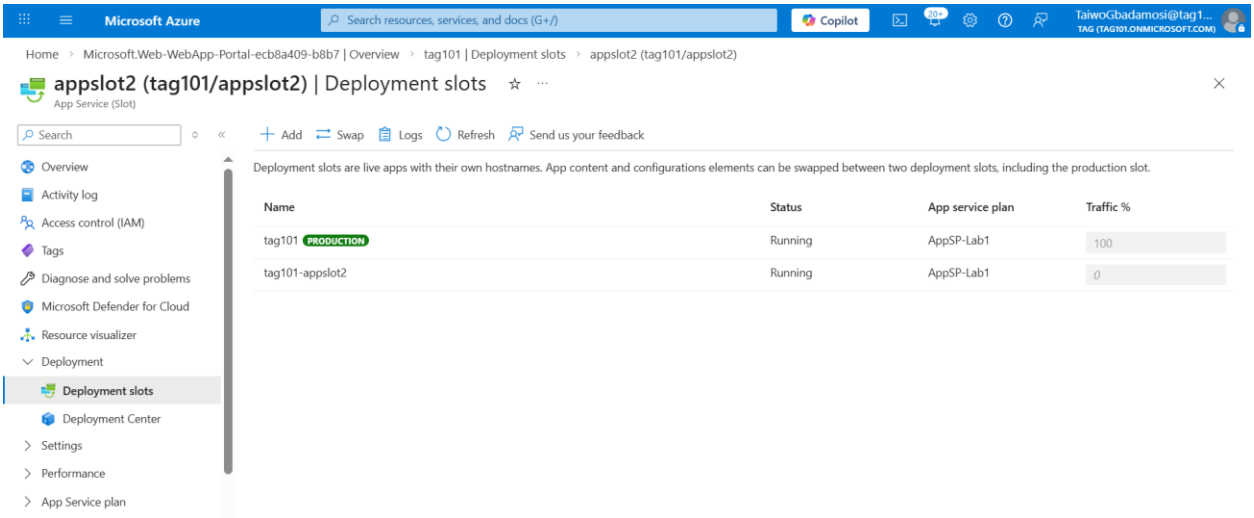
Section IV: Deployment Slots & Blue/Green Strategy

To simulate a professional DevOps workflow, I configured Deployment Slots.

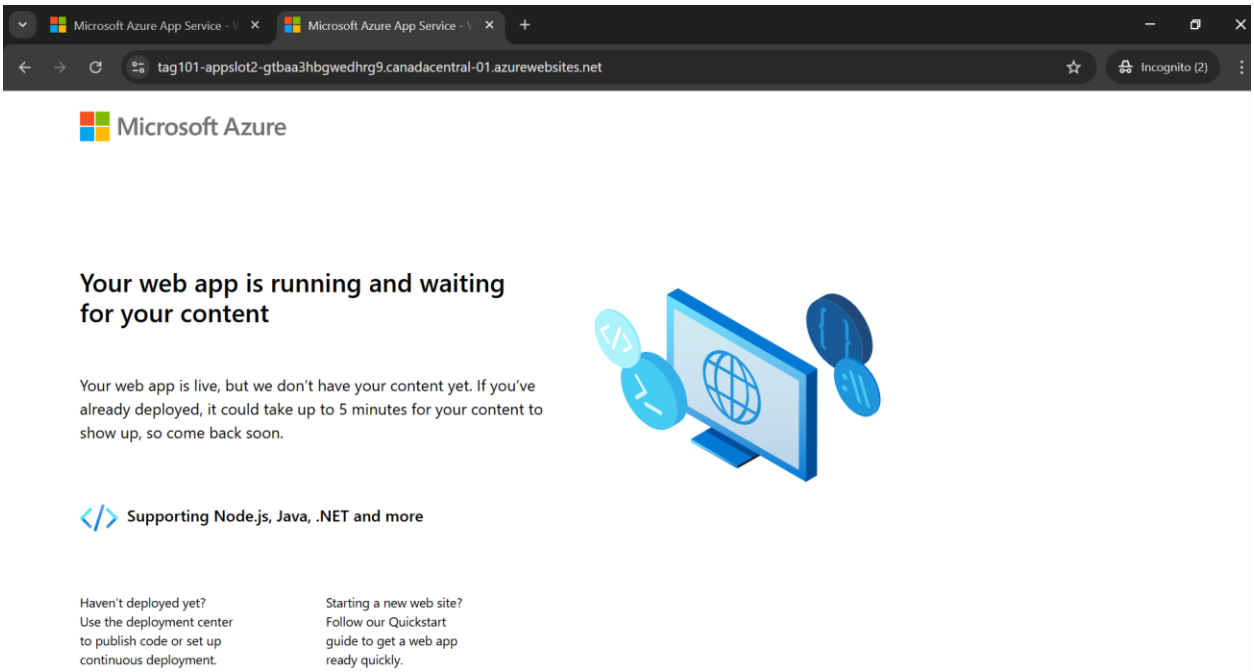
- **Staging vs. Production:** Created a Staging slot to host a new version of the app.
- **The Swap:** Executed a Swap operation, moving the staging code to production with zero downtime. This proves the ability to test code in a live-like environment before going public and this can also be used for seasonal web environment.



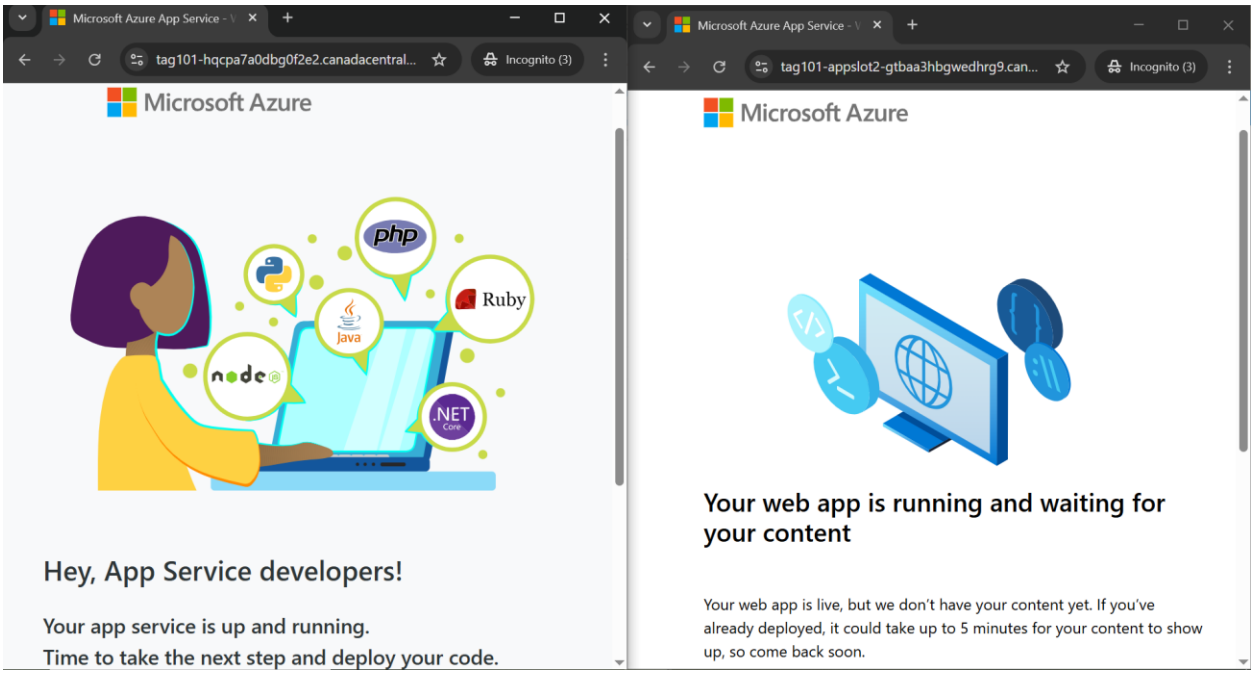
creating deployment slot within the app service web app



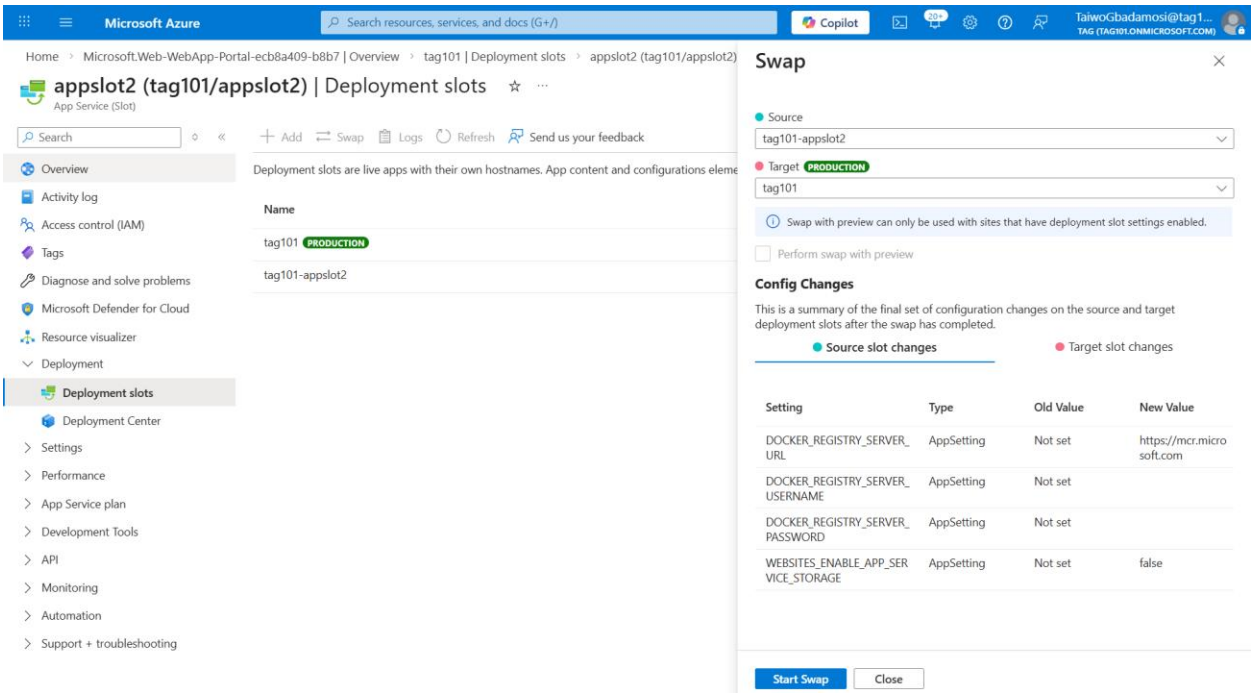
deployment slot successfully configured for the web app (as –appslot2 and 0 traffic assignment)



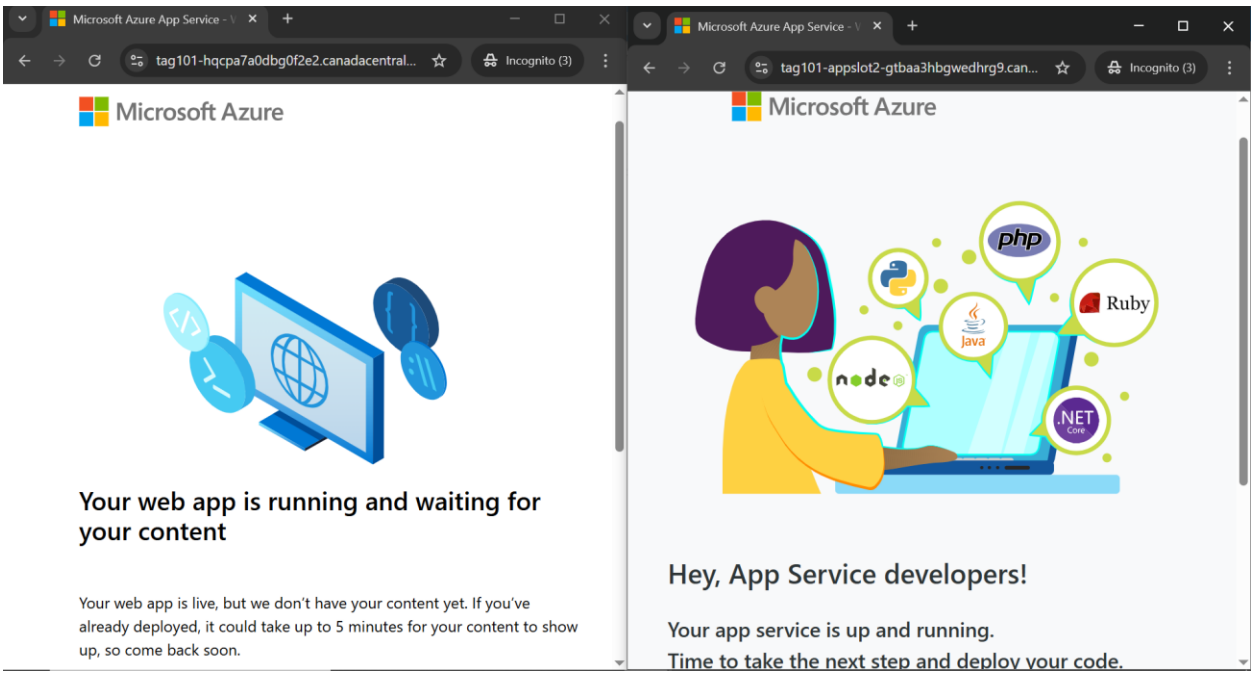
deployment slot resolvable from the internet



side-by-side shot of production slot and deployment slot



configuring swap operation to swap deployment slot with production slot

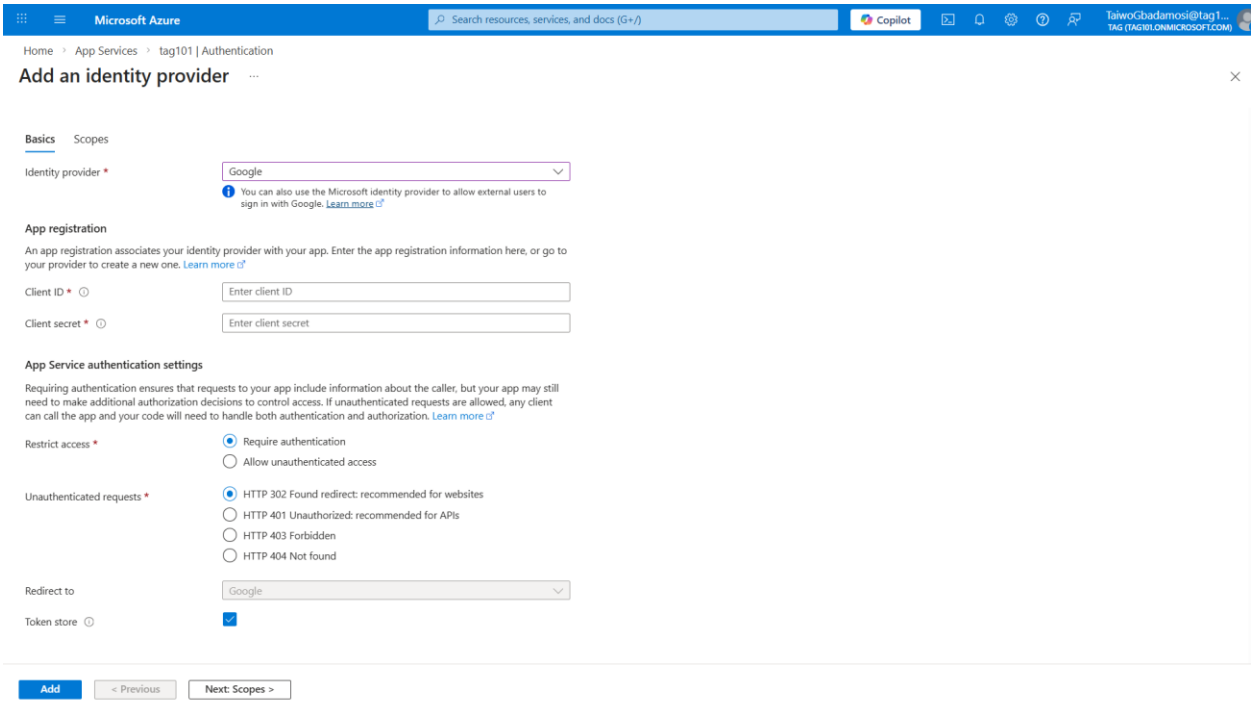


side-by-side shot of production slot and deployment slot showing successful execution of swap operation

Section V: Identity & Security Exploration

I explored the Authentication/Authorization blade for the Web App.

- **App Registration:** Researched the requirement for an App Registration in Microsoft Entra ID to enforce user logins.
- **Security Posture:** While not fully implemented, I analyzed how to restrict app access to only authenticated corporate users, a critical requirement for internal business tools.



scoping the implementation features of adding an identity provider for authentication purposes

4. Summary of Results

The project successfully demonstrated a robust, production-ready web stack. Traffic is efficiently managed via a Load Balancer for the IaaS layer, while the PaaS layer (Web Apps) provides high-speed deployment capabilities through slots and automated scaling to handle traffic spikes.

5. Lessons Learned & Professional Insights

- **Load Balancing:** Health Probes are vital for high availability. Without a correctly configured probe, a Load Balancer might send traffic to a crashed server.
- **DNS Latency vs. Internal Resolution:** Private DNS allows for instant internal routing even if the public domain hasn't propagated yet.

- **Zero-Downtime Swaps:** The Deployment Slots feature shows how modern enterprises can avoid the maintenance window by pre-warming staging slots and swapping them into production.
- **Scaling Efficiency:** Auto-scale rules must be carefully tuned (scale-up vs scale-down vs scale-in vs scale-out) to prevent situations where systems repeatedly scale in and out or up and down.
- **App Service Plan Density:** One app service plan can host multiple web apps, thereby saving costs in real world enterprise environments.